

SDA Projet 2: Arbres binaires de recherche

Pavlov Aleksandr s2400691

Table des matières

1	Pseudo-code et complexité de bstOptimalBuild	2
2	Implémentation de pdctCreate pour le BST 2D	2
3	Pseudo-code et complexité de bstRangeSearch	3
4	Comparaison empirique	3
5	Conclusion	5
6	Densité maximale de taxis à Porto	5

1 Pseudo-code et complexité de bstOptimalBuild

On trie les clés, puis on insère récursivement les médianes pour que l'arbre soit équilibré.

Algorithm 1 bstOptimalBuild(compfn, lkeys, lvalues)

```

1: pairs = tableau de paires clé et valeur
2: QuickSort(pairs, 0, N)
3: InsertMedians(bst, pairs, 0, N)
4: return bst

```

Algorithm 2 InsertMedians(*bst*, *pairs*, *left*, *right*)

```

1: if left ≥ right then return
2: end if
3: m = left + ⌊(right − left)/2⌋
4: bstInsert(bst, pairs[m])
5: InsertMedians(bst, pairs, left, m)
6: InsertMedians(bst, pairs, m + 1, right)

```

Complexité

- **Construction du tableau** : $O(N)$.
- **QuickSort** : meilleur et cas moyen $O(N \log N)$, pire cas $O(N^2)$.
- **InsertMedians** : N insertions, chacune en $O(\log N)$.

Cas	Complexité
Meilleur / moyen	$O(N \log N)$
Pire	$O(N^2)$

2 Implémentation de pdctCreate pour le BST 2D

Algorithm 3 BuildRecursive(*array*, *left*, *right*, *depth*)

```

1: if left ≥ right then return NULL
2: end if
3: m = left + ⌊(right − left)/2⌋
4: QuickSelect(array, left, right, m, depth)
5: node = nouveau node avec array[m]
6: node.left = BuildRecursive(array, left, m, depth + 1)
7: node.right = BuildRecursive(array, m + 1, right, depth + 1)
8: return node

```

Complexité

- **QuickSelect** meilleur et cas moyen $O(n)$, pire cas $O(n^2)$.

Cas	Complexité
Meilleur / moyen	$O(N \log N)$
Pire	$O(N^2)$

3 Pseudo-code et complexité de bstRangeSearch

Algorithm 4 bstRangeSearch(*bst*, *keymin*, *keymax*)

```

1: result = liste vide
2: bstRangeSearchRecursive(bst.root, keymin, keymax, result)
3: return result

```

Algorithm 5 bstRangeSearchRecursive(*node*, *keymin*, *keymax*, *result*)

```

1: if node = NULL then return
2: end if
3: if keymin ≤ node.key then
4:   bstRangeSearchRecursive(node.left, keymin, keymax, result)
5: end if
6: if keymin ≤ node.key ≤ keymax then
7:   result.append(node.value)
8: end if
9: if node.key < keymax then
10:  bstRangeSearchRecursive(node.right, keymin, keymax, result)
11: end if

```

Complexité

Soit k le nombre de clés dans $[keymin, keymax]$.

Cas	Complexité
Meilleur ($k = 0$, intervalle vide)	$O(\log N)$
Général	$O(\log N + k)$
Pire ($k = N$, tout l'arbre en range)	$O(N)$

4 Comparaison empirique

Variation de N ($r = 0,01$)

N	Structure	Construction	Exact ($\times 10^4$)	Boule ($\times 10^4$)
10^4	Liste	<0,001 s	0,281 s	0,343 s
	BST Morton	<0,001 s	<0,001 s	0,140 s
	BST 2D	<0,001 s	<0,001 s	<0,001 s
10^5	Liste	<0,001 s	3,45 s	6,06 s
	BST Morton	0,047 s	0,015 s	3,25 s
	BST 2D	0,062 s	0,016 s	0,093 s
10^6	Liste	<0,001 s	39,9 s	69,2 s
	BST Morton	0,484 s	0,015 s	42,2 s
	BST 2D	1,297 s	<0,001 s	0,734 s

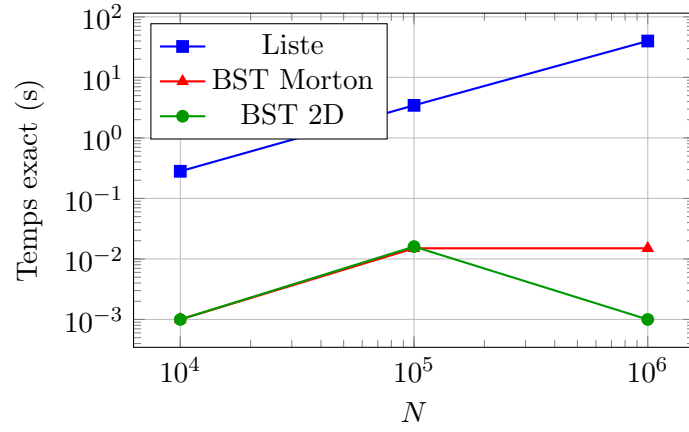


FIGURE 1 – Temps de recherche exacte en fonction de N (10^4 requêtes positives)

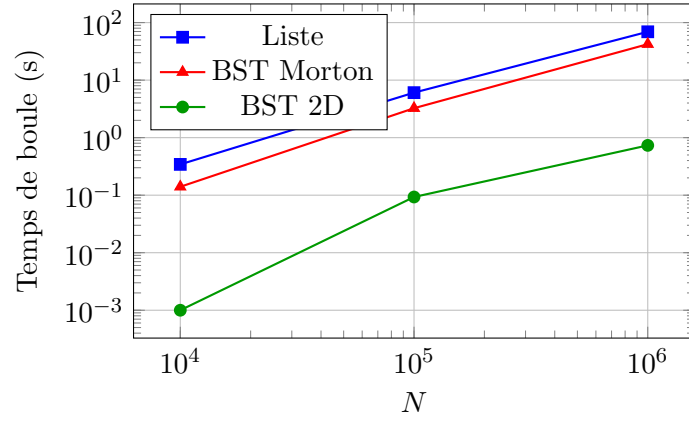


FIGURE 2 – Temps de recherche en boucle en fonction de N ($r = 0,01$, 10^4 requêtes)

Variation du rayon ($N = 10^5$)

r	Résultats moy.	Liste	BST Morton	BST 2D
0,01	31,1	6,06 s	3,25 s	0,093 s
0,05	751,7	6,00 s	21,3 s	1,52 s
0,10	2876	7,72 s	39,3 s	5,20 s

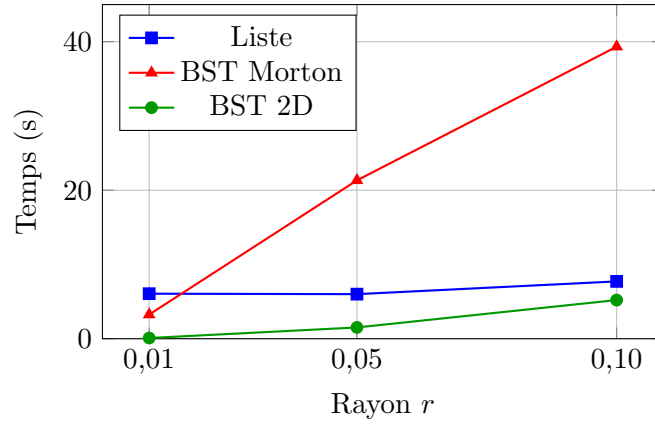


FIGURE 3 – Temps de recherche en boule en fonction du rayon ($N = 10^5, 10^4$ requêtes)

5 Conclusion

La recherche exacte confirme la théorie : la liste est en $O(N)$, les deux arbres en $O(\log N)$. À $N = 10^6$, la liste prend 39,9 s contre moins de 0,001 s pour le BST 2D.

Pour la recherche en boule, le résultat surprenant est que le BST Morton devient plus lent que la liste dès que le rayon augmente ($\times 5,1$ plus lent à $r = 0,10$). La raison : la plage $[Z_A, Z_B]$ contient de nombreux faux positifs, qui croissent plus vite que les vrais résultats quand r augmente.

Le BST 2D reste systématiquement le plus rapide grâce à son élagage par plan de séparation, efficace même pour de grands rayons. C'est donc l'implémentation à privilégier pour l'application taxi de Porto.

6 Densité maximale de taxis à Porto

Grille de 166×223 points espacés de 100 m. Pour chaque point, on compte les départs dans un rayon de 100 m.

Taxis max dans un rayon de 100 m	94 049
Longitude	$-8,5855^\circ$
Latitude	$41,1487^\circ$

La position correspond à la **gare de Campanhã**, le principal terminus ferroviaire de Porto. Les 94 049 départs représentent 5,5 % de l'ensemble des trajets, ce qui en fait l'emplacement idéal pour une nouvelle station de taxis.